

MecLib: Infobox-Bug & Zustandsbericht

2026-09-22 · @Someone

Die Ursache des Infobox-Bugs ist gefunden: ein Regressions-Verhalten in JSXGraph 1.12.2 selbst, kein Logikfehler in MecLib. Ein minimaler Ein-Zeilen-Fix in imeclib.js/meclib.js umgeht das Problem zuverlässig – siehe Lösungsvorschlag unten, der noch Deine Bestätigung braucht.

Diagnose

JSXGraph setzt bei Punkten standardmäßig `showInfobox: 'inherit'` (statt eines konkreten Booleans). Die interne Methode `evalVisProp('showinfobox')`, die diesen Wert auflöst, hat sich zwischen JSXGraph 1.4.4 (STACK 4.4-Ära) und 1.12.2 (aktuell von STACK verwendet) geändert: Trifft sie auf den String `'inherit'`, durchsucht sie seit 1.12.2 die `descendants` des Elements nach einem Kind-Objekt, dessen `inherits`-Liste das Element selbst enthält – und übernimmt dessen Wert, statt auf die Board-Voreinstellung zurückzufallen.

Das Problem: Jeder Punkt, der Endpunkt einer Linie/eines Pfeils/eines Segments ist (also praktisch jeder "interessante" Punkt in MecLib – Kraftpfeile, Momente, Verschiebungsvektoren, Federn, Dämpfer, Stäbe ...), hat genau ein solches Kind-Objekt: die anhängende Linie bzw. den Pfeil. Linien haben aber selbst kein `showinfobox`-Attribut, also liefert die Auflösung `undefined` statt `'inherit'`. In `board.updateInfobox()` führt `undefined` zum sofortigen Abbruch (`!vpsi` ist `true`) – die Infobox wird gar nicht erst angezeigt.

Ein freistehender Punkt ohne angehängte Linie (z. B. ein reiner `["point", ...]`) ist **nicht** betroffen, da er kein solches Kind-Objekt besitzt. Das erklärt, warum der Bug in der Praxis fast immer auftritt: In MecLib-Konstruktionen sind die meisten Punkte Endpunkte von Kraft-, Momenten- oder Verschiebungsvektoren.

Empirischer Nachweis: Ich habe lokal JSXGraph 1.12.2 sowie 1.4.4 installiert (`npm install jsxgraph@<version>`) und mit einem Playwright-Test denselben `initstring` (ein `point` und ein aktiver `force`) gegen `meclib.js` laufen lassen, jeweils mit simuliertem Hover über den Kraft-Endpunkt:

JSXGraph-Version	Hover über freistehenden Punkt	Hover über Kraft-Endpunkt (an Pfeil gebunden)
1.4.4	Infobox erscheint	Infobox erscheint

JSXGraph-Version	Hover über freistehenden Punkt	Hover über Kraft-Endpunkt (an Pfeil gebunden)
1.12.2	Infobox erscheint	Infobox bleibt unsichtbar

Damit ist bestätigt: Es handelt sich um eine echte Regression in JSXGraph selbst (in der generischen `evalVisProp`-Vererbungslogik), nicht um einen Fehler in MecLibs eigenem Code.

Vorgeschlagene Lösung

Korrektur: Meine erste Platzierung des Fixes (direkt nach der Board-Erzeugung, bei `board.infobox.distanceY`) war wirkungslos – `initBoard()` kopiert `JXG.Options` beim Aufruf in eine board-eigene Kopie, spätere Änderungen daran kommen zu spät. Das ist mir beim erneuten Testen aufgefallen; die folgende Fassung ist tatsächlich verifiziert.

Die Lösung bleibt ein Ein-Zeiler, muss aber **vor** `initBoard()` stehen – also bei den anderen `JXG.Options.point.*`-Voreinstellungen, die beide Dateien ohnehin schon dort setzen:

```
JXG.Options.point.showInfobox = true;
```

Betroffene Stellen (direkt nach `JXG.Options.point.highlight = false;`, kurz vor `// grid control`, jeweils deutlich vor dem `initBoard()`-Aufruf):

- `imeclib.js`
- `meclib.js`

Ich habe den Fix bereits in meinem lokalen Klon beider Dateien eingebaut und erneut gegen JSXGraph 1.12.2 verifiziert: aktiver Force-Endpunkt zeigt die Infobox korrekt, der "silent"-Fall (der die Infobox weiterhin absichtlich unterdrückt) funktioniert unverändert – in beiden Dateien.

Die beiden fertigen Dateien habe ich Dir im Chat als Download geschickt, damit Du sie wie geplant zuerst in einer echten STACK-Aufgabe testen kannst (externen Link temporär durch diese Version ersetzen). Wenn das dort passt, übernimmst Du die Änderung wie besprochen selbst in Deinem GitHub-Klon.

Versionskompatibilität des Fixes

Gute Frage – ich habe das systematisch nachgetestet, statt es nur anzunehmen. Ergebnis: **Der Fix ist auf allen getesteten Versionen unschädlich, eine Versionsweiche ist nicht nötig.**

Mit dem unveränderten `meclib.js` (ohne Fix) gegen neun JSXGraph-Versionen von 1.2.1 bis 1.13.3 getestet, zeigt sich: **Der Bug tritt ausschließlich in 1.12.1 und 1.12.2 auf** – alle Versionen davor und danach (JSXGraph hat es selbst in 1.13.0 wieder korrigiert) funktionieren bereits ohne unseren Fix korrekt:

JSXGraph-Version	Infobox bei verbundenem Punkt (ohne Fix)
1.2.1	OK
1.4.4	OK
1.9.2	OK
1.10.1	OK
1.11.1	OK
1.12.0	OK
1.12.1	fehlt (Bug)
1.12.2	fehlt (Bug) – aktuelle STACK-Version
1.13.0	OK (von JSXGraph selbst behoben)
1.13.1 – 1.13.3	OK

Zusätzlich mit **Fix** gegen 1.2.1, 1.4.4, 1.12.1, 1.12.2 und 1.13.3 getestet: Auf 1.2.1, 1.4.4 und 1.13.3 verhält sich alles – Hover auf freistehendem Punkt, verbundenem Punkt, und dem `state: "silent"` -Fall – exakt identisch zu ohne Fix (keine Konsolenfehler, byte-identische Infobox-Ausgaben). Auf 1.12.1 und 1.12.2 behebt der Fix den Bug wie erwartet.

Grund, warum das unbedenklich ist: Der Fix ersetzt lediglich den Default-Wert `'inherit'` durch den konkreten Wert `true` für `showInfobox`. Auf Versionen, auf denen `'inherit'` ohnehin korrekt zu `true` aufgelöst wird (alles außer 1.12.1/1.12.2), ist das Endergebnis identisch – es gibt keinen Verhaltensunterschied, den eine Versionsweiche absichern müsste.

Da `isNewerVersion()` in der Bibliothek bereits existiert, wäre eine Weiche trivial nachrüstbar, falls Du trotzdem lieber auf Nummer sicher gehen willst, z. B.:

```
if (isNewerVersion('1.12.0', JXG.version) && !isNewerVersion('1.12.2',
JXG.version)) {
  JXG.Options.point.showInfobox = true;
}
```

Meine Empfehlung ist aber, den Fix unbedingt bedingungslos zu setzen (wie im Abschnitt oben) – einfacher, und durch die Tests oben abgedeckt.

Allgemeiner Zustandsbericht

Grundstruktur, Dokumentation (Wiki mit über 100 Seiten, eine pro Objekttyp, plus Tutorials) und der ~35 Objektklassen umfassende Aufbau sind solide und gut nachvollziehbar. Ein paar Punkte sind mir beim Lesen aufgefallen:

Dateiduplikate. Im Root liegen `imeclib.js` (interaktiv) und `meclib.js` (statische Ableitung, laut Wiki manuell per "copy the generic part" gepflegt) als aktuelle, aktive Dateien – plus `imeclib.js.bak` und `meclib.js.bak`. Der Ordner `iMecLib/` mit einer weiteren `imeclib.js`-Kopie ist im eigenen Readme explizit als obsolet markiert. Falls die `.bak`-Dateien keine besondere Bedeutung haben, wären sie Kandidaten zum Aufräumen – sag mir gern, ob sie aus einem bestimmten Grund dort liegen.

Test-Lücke als Ursache für unentdeckte Regressionen. Der bisherige Workflow (laut `Updating-MecLib.md`) testet gegen drei jsfiddle-Seiten, die auf JSXGraph 1.2.1, 1.4.4 und einen Nightly-Build gepinnt sind – nicht aber gegen 1.12.2, die aktuelle STACK-Version. Genau das erklärt, warum diese Regression bislang nicht auffiel: Sie betrifft spezifisch 1.12.2 und spätere Versionen mit derselben `evalVisProp`-Logik.

Empfehlung: lokales Testsetup statt/zusätzlich zu jsfiddle. Da jsfiddle laut Dir zuletzt unzuverlässig war, habe ich hier bereits ein kleines lokales Setup aufgebaut: `npm install jsxgraph@<version>` plus eine minimale HTML-Seite mit den von MecLib erwarteten Globals (`mode`, `divid`, `initstring`, ...), automatisiert per Playwright (Mausbewegung simulieren, Konsole/Fehler mitschneiden, Board-Zustand direkt auspesen). Das lässt sich pro JSXGraph-Version in Sekunden aufsetzen und prüft exakt die Version, die STACK tatsächlich einsetzt – idealerweise ergänzend zu 1.2.1/1.4.4 auch 1.12.2 (und künftige STACK-Versionen) mit in die Testmatrix aufnehmen. Ich kann Dir dieses Setup gerne als eigenständiges kleines Repo/Skript übergeben, falls das nützlich ist.

Sonstiges. Der `History.md`-Changelog ist gepflegt und detailliert, das erleichtert Nachvollziehbarkeit sehr. Keine strukturellen Bedenken bezüglich der Single-File-Architektur (~100 KB) – für den Umfang gut wartbar, auch wenn ein Modul-Split langfristig die Navigation erleichtern könnte, falls die Datei weiter wächst.

Status und nächste Schritte

- **Fix:** In meinem lokalen Klon umgesetzt und gegen JSXGraph 1.12.2 verifiziert (siehe oben). Beide Dateien als Download geschickt – Du testest sie zunächst in einer echten STACK-Aufgabe (externen Link ersetzen) und übernimmst die Änderung bei Erfolg selbst in Deinem GitHub-Klon.
- **imeclib.js:** bleibt bestehen, wird nicht entfernt; bessere Kennzeichnung als obsolet wäre sinnvoll (z. B. ein Hinweis wie im `iMecLib/Readme.md`). Sag Bescheid, falls Du möchtest, dass ich dafür einen Vorschlag mache.
- **.bak -Dateien:** noch offen, ob sie bewusst dort liegen.
- **Lokale Testumgebung:** Anleitung als eigenständige Markdown-Datei `Local-Test-Environment.md` mitgeschickt, fertig zum Einfügen ins Wiki. Beschreibt Setup mit `npm install jsxgraph@<version>`, eine minimale Test-Seite (ohne Server, einfach im Browser öffnen) und optional automatisierte Playwright-Checks – inklusive des Skripts, mit dem der Infobox-Bug gefunden wurde.
- **JSXGraph-Issue:** noch nicht gemeldet, wie gewünscht erst nach Bestätigung, dass der Fix in STACK funktioniert.